

Physics-Informed Neural Networks (PINNs), Exact Automatic Differentiation, and Sobolev Space Regularization

A Unified Mathematical, Information-Theoretic, and Physical Foundation
for Solving Parametric Partial Differential Equations

Contents

1	The Physics-Informed Neural Network Paradigm	2
1.1	Forward vs. Inverse Problem Formulations	2
2	Automatic Differentiation and Composite Loss Mechanics	2
2.1	Automatic Differentiation vs. Numerical Differentiation	3
2.2	Composite Multi-Objective Loss Formulation	3
2.3	Soft vs. Hard Boundary Constraint Enforcements	4
3	Case Study: 1D Viscous Burgers' Equation	4
3.1	Derivation of the Residual Operator via AD	4
4	Advanced Optimization Dynamics and Gradient Pathology	4
4.1	Stiffness and Gradient Imbalance (NTK Analysis)	5
4.2	Adaptive Loss Weighting (Learning Rate Annealing)	5
4.3	Two-Stage Hybrid Optimization (Adam + L-BFGS)	5
5	Information-Theoretic and Physical Analogies	5
5.1	Physical Analogy: Principle of Least Action and Variational Mechanics	5
5.2	Information-Theoretic View: Maximum A Posteriori (MAP) in Sobolev Space	6
6	Production-Grade PyTorch Implementation	6

1 The Physics-Informed Neural Network Paradigm

Classical numerical methods for solving Partial Differential Equations (PDEs)—such as Finite Difference Methods (FDM), Finite Element Methods (FEM), and Finite Volume Methods (FVM)—rely on mesh generation and domain discretization. These methods suffer from the **Curse of Dimensionality** in high-dimensional domains, struggle with mesh generation for complex geometries, and cannot easily assimilate sparse, noisy empirical data into the PDE solution pipeline.

****Physics-Informed Neural Networks (PINNs)****, introduced by Raissi, Perdikaris, and Karniadakis (2019), represent a mesh-free paradigm that integrates physical laws directly into deep neural network architectures. By constraining the continuous neural network prediction $u_{\theta(\mathbf{x},t)}$ using automatic differentiation derivatives evaluated at discrete spatial-temporal collocation points, PINNs restrict the hypothesis space of the network strictly to functions that satisfy the governing differential equations.

Definition 1.1 (General Parameterized Non-Linear PDE). Let $\Omega \subset \mathbb{R}^d$ be a bounded spatial domain with boundary $\partial\Omega$, and let $t \in [0, T]$ represent the temporal domain. Consider a general non-linear parameterized PDE governing a continuous physical field $u(\mathbf{x}, t) : \Omega \times [0, T] \rightarrow \mathbb{R}^p$:

$$\mathcal{N}_{\mathbf{x},t}[u(\mathbf{x}, t); \boldsymbol{\lambda}] = 0, \quad \mathbf{x} \in \Omega, \quad t \in [0, T] \quad (1)$$

subject to boundary conditions (BCs) and initial conditions (ICs):

$$\mathcal{B}[u(\mathbf{x}, t)] = g(\mathbf{x}, t), \quad \mathbf{x} \in \partial\Omega, \quad t \in [0, T] \quad (2)$$

$$u(\mathbf{x}, 0) = h(\mathbf{x}), \quad \mathbf{x} \in \Omega \quad (3)$$

where $\mathcal{N}_{\mathbf{x},t}[\cdot; \boldsymbol{\lambda}]$ is a differential operator parameterized by physical constants $\boldsymbol{\lambda}$ (e.g., viscosity, thermal conductivity, elasticity), and $\mathcal{B}[\cdot]$ is a boundary operator (Dirichlet, Neumann, or Robin).

In the PINN framework, the continuous solution $u(\mathbf{x}, t)$ is approximated by a deep surrogate neural network $u_{\theta(\mathbf{x},t)}$ parameterized by trainable weights and biases $\boldsymbol{\theta}$.

1.1 Forward vs. Inverse Problem Formulations

PINNs provide a unified architecture capable of seamlessly addressing both forward and inverse physical modeling problems:

1. **Forward Problems (PDE Solving):** Given known PDE parameters $\boldsymbol{\lambda}$, boundary conditions $g(\mathbf{x}, t)$, and initial conditions $h(\mathbf{x})$, optimize network parameters $\boldsymbol{\theta}$ to construct a continuous surrogate solver $u_{\theta(\mathbf{x},t)}$ over the entire spatiotemporal domain $\Omega \times [0, T]$.
2. **Inverse Problems (Parameter Identification):** Given sparse, noisy experimental measurements of the field $\{(\mathbf{x}_i, t_i, u_i^*)\}_{i=1}^{N_u}$ alongside the structural form of operator $\mathcal{N}_{\mathbf{x},t}$, jointly optimize both network weights $\boldsymbol{\theta}$ and unknown physical constants $\boldsymbol{\lambda}$ simultaneously:

$$(\boldsymbol{\theta}^*, \boldsymbol{\lambda}^*) = \arg \min_{\boldsymbol{\theta}, \boldsymbol{\lambda}} \mathcal{L}_{\text{PINN}}(\boldsymbol{\theta}, \boldsymbol{\lambda}) \quad (4)$$

2 Automatic Differentiation and Composite Loss Mechanics

Computing higher-order spatial and temporal derivatives ($\frac{\partial u}{\partial t}, \nabla_{\mathbf{x}} u, \Delta_{\mathbf{x}} u$) accurately is the central requirement of physics-informed learning.

2.1 Automatic Differentiation vs. Numerical Differentiation

Standard numerical approximations (e.g., finite differences $\frac{\partial u}{\partial x} \approx \frac{u(x+\Delta x) - u(x)}{\Delta x}$) introduce truncation errors $\mathcal{O}(\Delta x^2)$ and grid-dependent discretization artifacts. Symbolic differentiation generates exponentially large expressions (**expression swell**).

PINNs leverage **Reverse-Mode Automatic Differentiation (AD)**. By applying the multivariate chain rule backwards across the exact computational graph of the neural network $u_{\theta}(\mathbf{x}, t)$, AD evaluates partial derivatives of u_{θ} with respect to input coordinates $(\mathbf{x}, t)$ to machine precision without grid discretization:

$$\frac{\partial^k u_{\theta}}{\partial x_j^k} = \text{Autograd} \left(u_{\theta}(\mathbf{x}, t), x_j, \text{order}=k \right) \quad (5)$$

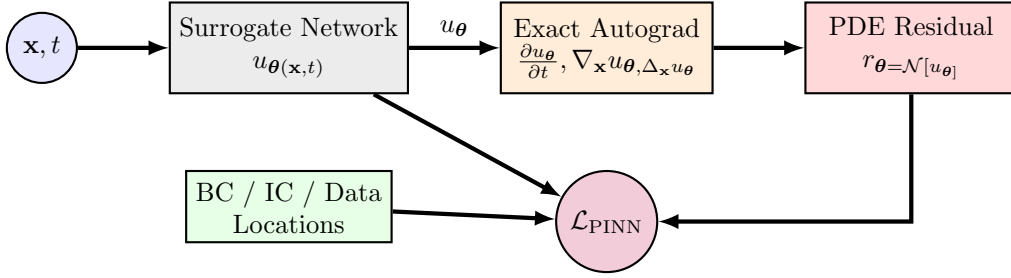


Figure 1: Computational graph architecture of a Physics-Informed Neural Network (PINN). Inputs pass through surrogate network u_{θ} , exact AD evaluates partial derivatives, and physical PDE residuals assemble into a composite loss.

2.2 Composite Multi-Objective Loss Formulation

PINNs replace explicit structural equality constraints with a penalized **Composite Loss Function** evaluated over discrete point sets across the domain.

Definition 2.1 (PINN Composite Loss Function). Let $\mathcal{S}_{\text{pde}} = \{(\mathbf{x}_f^{(i)}, t_f^{(i)})\}_{i=1}^{N_f} \subset \Omega \times [0, T]$ be N_f interior **collocation points**. Let $\mathcal{S}_{\text{bc}} = \{(\mathbf{x}_b^{(i)}, t_b^{(i)}, g_i)\}_{i=1}^{N_b} \subset \partial\Omega \times [0, T]$ be N_b boundary points, $\mathcal{S}_{\text{ic}} = \{(\mathbf{x}_0^{(i)}, 0, h_i)\}_{i=1}^{N_0} \subset \Omega \times \{0\}$ be N_0 initial points, and $\mathcal{S}_{\text{data}} = \{(\mathbf{x}_d^{(i)}, t_d^{(i)}, u_d^{(i)})\}_{i=1}^{N_d}$ be N_d sparse sensor measurements.

The total composite loss $\mathcal{L}_{\text{PINN}}(\theta)$ is the weighted sum of mean squared residuals:

$$\mathcal{L}_{\text{PINN}}(\theta) = w_f \mathcal{L}_{\text{pde}}(\theta) + w_b \mathcal{L}_{\text{bc}}(\theta) + w_0 \mathcal{L}_{\text{ic}}(\theta) + w_d \mathcal{L}_{\text{data}}(\theta) \quad (6)$$

where $w_f, w_b, w_0, w_d > 0$ are loss penalty weights, and the individual loss terms are:

$$\mathcal{L}_{\text{pde}}(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} \left\| \mathcal{N}_{\mathbf{x}, t} \left[u_{\theta}(\mathbf{x}_f^{(i)}, t_f^{(i)}); \lambda \right] \right\|_2^2 \quad (7)$$

$$\mathcal{L}_{\text{bc}}(\theta) = \frac{1}{N_b} \sum_{i=1}^{N_b} \left\| \mathcal{B} \left[u_{\theta}(\mathbf{x}_b^{(i)}, t_b^{(i)}) \right] - g(\mathbf{x}_b^{(i)}, t_b^{(i)}) \right\|_2^2 \quad (8)$$

$$\mathcal{L}_{\text{ic}}(\theta) = \frac{1}{N_0} \sum_{i=1}^{N_0} \left\| u_{\theta}(\mathbf{x}_0^{(i)}, 0) - h(\mathbf{x}_0^{(i)}) \right\|_2^2 \quad (9)$$

$$\mathcal{L}_{\text{data}}(\theta) = \frac{1}{N_d} \sum_{i=1}^{N_d} \left\| u_{\theta}(\mathbf{x}_d^{(i)}, t_d^{(i)}) - u_d^{(i)} \right\|_2^2 \quad (10)$$

2.3 Soft vs. Hard Boundary Constraint Enforcements

In the composite loss formulation, boundary and initial conditions are enforced ****softly**** as penalty terms ($\mathcal{L}_{bc}, \mathcal{L}_{ic}$), meaning boundary constraints are satisfied only approximately.

Alternatively, boundary conditions can be enforced ****hard**** (exactly) by reparameterizing the output layer via an ****Ansatz transformation****:

$$u_{\theta^{\text{hard}}}(\mathbf{x}, t) = B(\mathbf{x}, t) + A(\mathbf{x}, t) \cdot \mathcal{N}_{\theta(\mathbf{x}, t)} \quad (11)$$

where $B(\mathbf{x}, t)$ is a distance function satisfying boundary conditions exactly, and $A(\mathbf{x}, t)$ vanishes on $\partial\Omega \times [0, T]$ ($A|_{\partial\Omega} = 0$). Hard constraints guarantee $\mathcal{L}_{bc}(\theta) \equiv 0$ by construction.

3 Case Study: 1D Viscous Burgers' Equation

To illustrate the mathematical workflow, consider the classical 1D ****Viscous Burgers' Equation****, a fundamental non-linear PDE in fluid dynamics capturing convective transport, shock formation, and viscous diffusion.

Definition 3.1 (1D Viscous Burgers' Equation). Find $u(x, t)$ over $x \in [-1, 1]$ and $t \in [0, T]$ satisfying:

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} - \nu \frac{\partial^2 u}{\partial x^2} = 0, \quad x \in [-1, 1], \quad t \in [0, T] \quad (12)$$

subject to Dirichlet boundary conditions and non-smooth initial condition:

$$u(-1, t) = 0, \quad u(1, t) = 0, \quad t \in [0, T] \quad (13)$$

$$u(x, 0) = -\sin(\pi x), \quad x \in [-1, 1] \quad (14)$$

where $\nu > 0$ is the fluid kinematic viscosity coefficient (e.g., $\nu = \frac{0.01}{\pi}$).

3.1 Derivation of the Residual Operator via AD

Let $u_{\theta(x,t)}$ be a deep neural network surrogate. The PDE residual function $r_{\theta(x,t)}$ is defined as:

$$r_{\theta(x,t)} \equiv \frac{\partial u_{\theta}}{\partial t}(x, t) + u_{\theta(x,t)} \frac{\partial u_{\theta}}{\partial x}(x, t) - \nu \frac{\partial^2 u_{\theta}}{\partial x^2}(x, t) \quad (15)$$

Evaluating partial derivatives explicitly via reverse-mode AD chain rules: . First temporal derivative: $\frac{\partial u_{\theta}}{\partial t} = \text{Autograd}(u_{\theta, t})$

First spatial derivative: $\frac{\partial u_{\theta}}{\partial x} = \text{Autograd}(u_{\theta, x})$

Second spatial derivative: $\frac{\partial^2 u_{\theta}}{\partial x^2} = \text{Autograd}\left(\frac{\partial u_{\theta}}{\partial x}, x\right)$

The PDE collocation loss evaluates over N_f sampled points $\{(x_f^{(i)}, t_f^{(i)})\}_{i=1}^{N_f}$:

$$\mathcal{L}_{\text{pde}}(\theta) = \frac{1}{N_f} \sum_{i=1}^{N_f} \left| \frac{\partial u_{\theta}}{\partial t}(x_f^{(i)}, t_f^{(i)}) + u_{\theta}(x_f^{(i)}, t_f^{(i)}) \frac{\partial u_{\theta}}{\partial x}(x_f^{(i)}, t_f^{(i)}) - \nu \frac{\partial^2 u_{\theta}}{\partial x^2}(x_f^{(i)}, t_f^{(i)}) \right|^2 \quad (16)$$

4 Advanced Optimization Dynamics and Gradient Pathology

Training PINNs presents unique non-convex optimization challenges not encountered in standard supervised classification.

4.1 Stiffness and Gradient Imbalance (NTK Analysis)

Wang, Yu, and Perdikaris (2021) analyzed PINN training dynamics using **Neural Tangent Kernel (NTK)** theory. The composite loss involves competing objectives whose gradient magnitudes can differ by orders of magnitude:

$$\|\nabla_{\theta} \mathcal{L}_{\text{pde}}\|_2 \ll \|\nabla_{\theta} \mathcal{L}_{\text{bc}}\|_2 \quad (17)$$

This discrepancy causes **stiffness in gradient flow**: gradients from boundary/initial losses dominate, causing the network to fit boundary points while leaving PDE interior residuals unresolved.

4.2 Adaptive Loss Weighting (Learning Rate Annealing)

To balance multi-objective gradient magnitudes dynamically, adaptive weighting rescales penalty parameters w_f, w_b, w_0 at iteration k :

Definition 4.1 (Learning Rate Annealing for PINNs). Let $\hat{w}_b^{(k)}$ be the candidate weight for the boundary loss term at step k :

$$\hat{w}_b^{(k)} = \frac{\max_{\theta} |\nabla_{\theta} \mathcal{L}_{\text{pde}}(\theta^{(k)})|}{|\nabla_{\theta} \mathcal{L}_{\text{bc}}(\theta^{(k)})|} \quad (18)$$

Update loss weight $w_b^{(k+1)}$ using an exponential moving average ($\alpha = 0.9$):

$$w_b^{(k+1)} = \alpha w_b^{(k)} + (1 - \alpha) \hat{w}_b^{(k)} \quad (19)$$

4.3 Two-Stage Hybrid Optimization (Adam + L-BFGS)

First-order stochastic gradient optimizers (Adam) excel at rapidly escaping random initializations and finding coarse structural basins. However, they struggle to converge to tight tolerances ($< 10^{-6}$) required for PDE residual minimization.

Production PINN pipelines utilize a **Two-Stage Hybrid Optimization Strategy**: . **Stage 1 (Adam Pre-training)**: Run 10,000–20,000 iterations of Adam optimizer ($\eta = 10^{-3}$) to establish global field structures.

Stage 2 (L-BFGS Fine-tuning): Transition to **L-BFGS (Limited-memory Broyden-Fletcher-Goldfarb-Shanno)**, a second-order quasi-Newton optimizer that approximates the inverse Hessian matrix $\mathbf{H}^{-1}$. L-BFGS achieves superlinear convergence, driving PDE residuals down to machine precision.

5 Information-Theoretic and Physical Analogies

5.1 Physical Analogy: Principle of Least Action and Variational Mechanics

In classical field theory and analytical mechanics, physical trajectories satisfy Hamilton’s **Principle of Least Action**:

$$\delta S = \delta \int_{t_1}^{t_2} \mathcal{L}_{\text{Lagrangian}}(q, \dot{q}, t) dt = 0 \quad (20)$$

Minimizing the PINN PDE residual loss $\mathcal{L}_{\text{pde}}(\theta)$ corresponds directly to finding a field configuration $u_{\theta}(\mathbf{x}, t)$ that minimizes the integrated squared Euler-Lagrange residual over the spacetime domain:

$$S_{\text{residual}}[u_{\theta}] = \int_{\Omega \times [0, T]} |\mathcal{N}_{\mathbf{x}, t}[u_{\theta}]|^2 d\mathbf{x} dt \quad (21)$$

The neural network acts as a continuous trial function space parameterizing variational field states.

5.2 Information-Theoretic View: Maximum A Posteriori (MAP) in Sobolev Space

From a Bayesian information-theoretic perspective, training a standard data-driven network evaluates a Maximum Likelihood Estimate (MLE) over empirical data $P(\mathcal{D} | \theta)$.

PINNs perform **Maximum A Posteriori (MAP)** estimation in continuous **Sobolev Spaces** $W^{k,p}(\Omega)$:

$$\mathcal{P}(\theta | \mathcal{D}, \text{Physics}) \propto \underbrace{P(\mathcal{D} | \theta)}_{\text{Data Likelihood}} \cdot \underbrace{P(\theta | \text{PDE})}_{\text{Physical Prior}} \quad (22)$$

The PDE residual loss acts as a functional prior probability density:

$$P(\theta | \text{PDE}) \propto \exp \left(-\gamma \int_{\Omega \times [0, T]} |\mathcal{N}[u_\theta]|^2 dx dt \right) \quad (23)$$

This physical prior regularizes the hypothesis space, restricting solutions to Sobolev manifolds that strictly satisfy physical conservation laws.

6 Production-Grade PyTorch Implementation

The following complete, production-grade PyTorch script implements a Physics-Informed Neural Network (PINN) for solving the 1D Viscous Burgers' Equation. It features exact higher-order automatic differentiation ('torch.autograd.grad'), composite PDE/BC/IC loss evaluation, collocation sampling, Adam pre-training, and L-BFGS second-order optimization.

```

1 import torch
2 import torch.nn as nn
3 import numpy as np
4
5 # Set random seeds and precision defaults for physical reproducibility
6 torch.manual_seed(42)
7 np.random.seed(42)
8 torch.set_default_dtype(torch.float64)
9
10 class PhysicsInformedNN(nn.Module):
11     """
12     Production-grade Physics-Informed Neural Network (PINN) Surrogate
13     Architecture.
14     Solves the 1D Viscous Burgers' Equation: u_t + u * u_x - nu * u_xx = 0
15     """
16     def __init__(self, layers: list, nu: float = 0.01 / np.pi):
17         super(PhysicsInformedNN, self).__init__()
18         self.nu = nu
19         self.depth = len(layers) - 1
20
21         # Build fully connected MLP backbone with Tanh activations
22         layer_list = []
23         for i in range(self.depth):
24             layer_list.append(nn.Linear(layers[i], layers[i+1]))
25             if i < self.depth - 1:
26                 layer_list.append(nn.Tanh())
27
28         self.net = nn.Sequential(*layer_list)
29         self._initialize_weights()
30
31     def _initialize_weights(self):
32         """Xavier/Glorot Normal Initialization for smooth derivative
33         propagation."""

```

```

32     for m in self.net.modules():
33         if isinstance(m, nn.Linear):
34             nn.init.xavier_normal_(m.weight)
35             nn.init.zeros_(m.bias)
36
37     def forward(self, x: torch.Tensor, t: torch.Tensor) -> torch.Tensor:
38         """Evaluates continuous solution surrogate  $u_\theta(x, t)$ ."""
39         inputs = torch.cat([x, t], dim=1)
40         return self.net(inputs)
41
42     def compute_pde_residual(self, x: torch.Tensor, t: torch.Tensor) ->
43     torch.Tensor:
44         """
45         Computes the exact PDE residual via Higher-Order Reverse-Mode
46         Automatic Differentiation:
47          $r = u_t + u * u_x - \nu * u_{xx}$ 
48         """
49         # Ensure input tensors track gradients for autograd computation
50         x.requires_grad_(True)
51         t.requires_grad_(True)
52
53         u = self.forward(x, t)
54
55         # First-order partial derivatives:  $u_t, u_x$ 
56         grads = torch.autograd.grad(
57             outputs=u,
58             inputs=[x, t],
59             grad_outputs=torch.ones_like(u),
60             create_graph=True,
61             retain_graph=True
62         )
63         u_x = grads[0]
64         u_t = grads[1]
65
66         # Second-order spatial partial derivative:  $u_{xx}$ 
67         u_xx = torch.autograd.grad(
68             outputs=u_x,
69             inputs=x,
70             grad_outputs=torch.ones_like(u_x),
71             create_graph=True,
72             retain_graph=True
73         ) [0]
74
75         # PDE Residual:  $r(x, t) = u_t + u * u_x - \nu * u_{xx}$ 
76         residual = u_t + u * u_x - self.nu * u_xx
77         return residual
78
79 class PINNTrainer:
80     """
81     PINN Optimization Pipeline handling multi-objective composite losses,
82     collocation point sampling, Adam pre-training, and L-BFGS fine-tuning.
83     """
84     def __init__(self, model: PhysicsInformedNN,
85                  N_0: int = 100, N_b: int = 100, N_f: int = 10000):
86         self.model = model
87         self.N_0 = N_0
88         self.N_b = N_b
89         self.N_f = N_f

```

```

90         self._generate_collocation_dataset()
91
92     def _generate_collocation_dataset(self):
93         """Samples spatial-temporal collocation points across domain, BCs,
94         and ICs."""
95         # 1. Initial Conditions (t = 0, x in [-1, 1]): u(x, 0) = -sin(pi *
96         x)
97         x_0 = np.random.uniform(-1.0, 1.0, (self.N_0, 1))
98         t_0 = np.zeros((self.N_0, 1))
99         u_0 = -np.sin(np.pi * x_0)
100
101         # 2. Dirichlet Boundary Conditions (x = -1 and x = 1, t in [0, 1]):
102         u(-1, t) = u(1, t) = 0
103         t_b = np.random.uniform(0.0, 1.0, (self.N_b, 1))
104         x_b_left = -np.ones((self.N_b // 2, 1))
105         x_b_right = np.ones((self.N_b // 2, 1))
106         x_b = np.vstack([x_b_left, x_b_right])
107         t_b = np.vstack([t_b[:self.N_b // 2], t_b[self.N_b // 2:]])
108         u_b = np.zeros((self.N_b, 1))
109
110         # 3. Interior PDE Collocation Points (x in [-1, 1], t in [0, 1])
111         x_f = np.random.uniform(-1.0, 1.0, (self.N_f, 1))
112         t_f = np.random.uniform(0.0, 1.0, (self.N_f, 1))
113
114         # Convert to PyTorch Tensors
115         self.x_0 = torch.tensor(x_0, dtype=torch.float64)
116         self.t_0 = torch.tensor(t_0, dtype=torch.float64)
117         self.u_0 = torch.tensor(u_0, dtype=torch.float64)
118
119         self.x_b = torch.tensor(x_b, dtype=torch.float64)
120         self.t_b = torch.tensor(t_b, dtype=torch.float64)
121         self.u_b = torch.tensor(u_b, dtype=torch.float64)
122
123         self.x_f = torch.tensor(x_f, dtype=torch.float64)
124         self.t_f = torch.tensor(t_f, dtype=torch.float64)
125
126     def loss_function(self) -> torch.Tensor:
127         """Evaluates composite loss: L_PINN = L_pde + L_ic + L_bc."""
128         # Initial Condition Loss
129         u_0_pred = self.model(self.x_0, self.t_0)
130         loss_ic = torch.mean((u_0_pred - self.u_0)**2)
131
132         # Boundary Condition Loss
133         u_b_pred = self.model(self.x_b, self.t_b)
134         loss_bc = torch.mean((u_b_pred - self.u_b)**2)
135
136         # Interior PDE Residual Loss
137         residual = self.model.compute_pde_residual(self.x_f, self.t_f)
138         loss_pde = torch.mean(residual**2)
139
140         total_loss = loss_pde + loss_ic + loss_bc
141         return total_loss
142
143     def train(self, adam_epochs: int = 5000, lbfgs_max_iter: int = 1000):
144         """Executes Hybrid Optimization Pipeline: Adam Pre-training + L-
145         BFGS Convergence."""
146         # --- Stage 1: Adam Optimizer ---
147         optimizer_adam = torch.optim.Adam(self.model.parameters(), lr=1e-3)
148         print("== STAGE 1: ADAM PRE-TRAINING INITIALIZED ==")

```



```

146         for epoch in range(1, adam_epochs + 1):
147             optimizer_adam.zero_grad()
148             loss = self.loss_function()
149             loss.backward()
150             optimizer_adam.step()
151
152             if epoch == 1 or epoch % 1000 == 0:
153                 print(f"Adam Epoch [{epoch:05d}/{adam_epochs}] - Total PINN
Loss: {loss.item():.6e}")
154
155         # --- Stage 2: L-BFGS Quasi-Newton Fine-Tuning ---
156         print("\n=== STAGE 2: L-BFGS SECOND-ORDER FINE-TUNING INITIALIZED
===")
157         optimizer_lbfgs = torch.optim.LBFGS(
158             self.model.parameters(),
159             max_iter=lbfgs_max_iter,
160             tolerance_grad=1e-7,
161             tolerance_change=1e-9,
162             history_size=50,
163             line_search_fn="strong_wolfe"
164         )
165
166         step_count = [0]
167         def closure():
168             optimizer_lbfgs.zero_grad()
169             loss = self.loss_function()
170             loss.backward()
171             step_count[0] += 1
172             if step_count[0] % 100 == 0:
173                 print(f"L-BFGS Step [{step_count[0]:04d}] - Total PINN Loss
: {loss.item():.6e}")
174             return loss
175
176         optimizer_lbfgs.step(closure)
177         print(f"Final Converged Loss: {self.loss_function().item():.6e}")
178
179
180 if __name__ == "__main__":
181     # Define Network Architecture: 2 Inputs (x, t) -> 8 Hidden Layers of 20
Neurons -> 1 Output (u)
182     network_layers = [2, 20, 20, 20, 20, 20, 20, 20, 20, 20, 1]
183     viscosity_nu = 0.01 / np.pi
184
185     pinn_model = PhysicsInformedNN(layers=network_layers, nu=viscosity_nu)
186     trainer = PINNTrainer(model=pinn_model, N_0=100, N_b=100, N_f=2000)
187
188     print("=== PHYSICS-INFORMED NEURAL NETWORK (PINN) INITIALIZATION
SUCCESSFUL ===")
189     print(f"Network Depth           : {len(network_layers) - 1} layers")
190     print(f"Viscosity Parameter : nu = {viscosity_nu:.6f}")
191     print(f"Collocation Points   : N_f = {trainer.N_f}, N_b = {trainer.N_b
}, N_0 = {trainer.N_0}\n")
192
193     # Run Hybrid Training Loop
194     trainer.train(adam_epochs=2000, lbfgs_max_iter=500)
195
196     # Evaluate Model on Test Point
197     x_test = torch.tensor([[0.5]], dtype=torch.float64)
198     t_test = torch.tensor([[0.25]], dtype=torch.float64)
199     u_pred = pinn_model(x_test, t_test).item()

```

```
200     res_pred = pinn_model.compute_pde_residual(x_test, t_test).item()
201
202     print(f"\nTest Evaluation at (x=0.5, t=0.25):")
203     print(f"Predicted Field Value u(0.5, 0.25) : {u_pred:.6f}")
204     print(f"Evaluated PDE Residual r(0.5, 0.25): {res_pred:.6e}")
```

Listing 1: Production-grade PyTorch implementation of a Physics-Informed Neural Network (PINN) for 1D Burgers' Equation.